

Flow-Conditioned Metric 3D Point Tracking

Optical flow + metric depth + a DINOv3-conditioned Mamba-3 depth refiner

Masahiro Ogawa

July 15, 2026

Abstract

We address tracking query points in *metric* 3D across a monocular video, where the quantity that ultimately governs real-world accuracy is the per-point depth. Rather than learning tracking end-to-end, we compose two frozen, well-understood front-ends — dense optical flow [1, 2] for 2D correspondence and a monocular metric-depth network [3] for the third dimension — and learn only the residual they cannot supply: the depth along each point’s (fixed) pixel ray, refined by a tiny causal Mamba-3 state-space model [4] conditioned on DINOv3 [5] appearance features. This isolates the single degree of freedom that determines 3D error, while keeping the model small (~ 0.5 M parameters) and the pipeline interpretable. On the TAPVid-3D [6] minimal benchmark, our best configuration (WAFT flow + Depth-Anything-3 depth + the DINOv3-conditioned refiner) attains the highest *absolute* metric accuracy of all methods evaluated under identical conditions — mean metric-AJ 0.246, ahead of strong feed-forward 3D trackers — confirming that a targeted depth-only refinement recovers real-world 3D structure that scale-invariant training leaves on the table. To measure this we introduce an absolute-metric evaluation protocol that scores error at fixed metre thresholds, exposing depth quality that the standard scale-invariant leaderboard metric hides. A companion reliability analysis — reproduced with each competitor’s own official evaluator — explains why several published 3D trackers collapse under a fixed monocular, single-GPU budget, separating genuine method limitations from evaluation artefacts.

Contents

1	Method	2
1.1	Overview	2
1.2	2D track: SEA-RAFT flow-chaining (baseline, frozen)	2
1.3	Metric depth and differentiable unprojection	2
1.4	v33: Mamba-3 depth-along-ray refiner (the method)	3
1.5	Why depth-only (and not 2D)	4
1.6	v35: image-conditioned joint 2D–depth refiner	4
1.7	v36: bidirectional flow	4
1.8	v37–v39: WAFT optical-flow front-end	5
1.9	v40: WAFT + DA3-g metric depth (no refiner)	5
1.10	v41: DA3-g depth + vmamba3 refiner	7
1.11	v42: world-frame tracking with ego-motion/object-motion decomposition (proposed)	7
1.12	Training	9
2	Evaluation	9
2.1	Protocol and metrics	9
2.2	Leaderboard (scale-invariant) results	10
2.3	Absolute-metric results: the reversal	11
2.4	Comparison to state-of-the-art	12

2.5 Qualitative results	14
2.6 Efficiency	15

1 Method

1.1 Overview

Given a video $\{I_t\}_{t=1}^F$ and query points $\{(\mathbf{u}_n, a_n)\}_{n=1}^N$ (pixel position $\mathbf{u}_n$ at anchor frame a_n), the goal is a 3D track $\hat{\mathbf{X}} \in \mathbb{R}^{F \times N \times 3}$ and visibility $\hat{v} \in \{0, 1\}^{F \times N}$. Our pipeline (Fig. 1) has a *training-free* baseline — SEA-RAFT flow-chaining for the 2D track plus DA3-l unprojection for 3D — and a small *learned* module, v33, that refines depth.

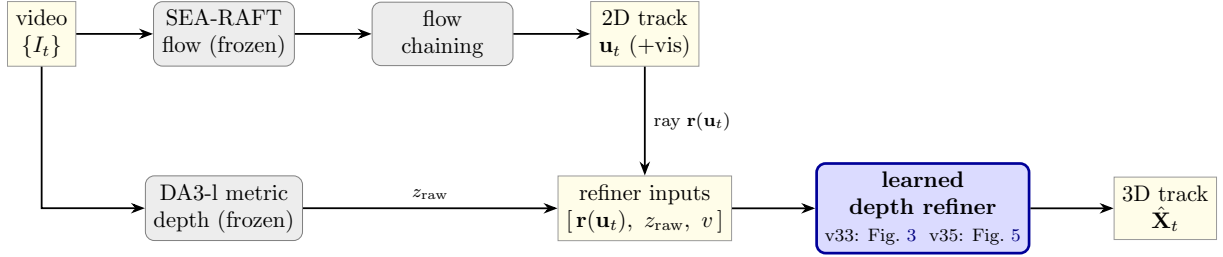


Figure 1: Overall pipeline. Grey = frozen / training-free; the **large blue box** is the only learned module (0.40M params); its internals are Fig. 3 (v33) and Fig. 5 (v35). SEA-RAFT’s 2D track $\mathbf{u}_t$ (and visibility) are *never* modified. The refiner’s *inputs* (yellow box) — the frozen pixel ray $\mathbf{r}(\mathbf{u}_t)$ (from $\mathbf{u}_t$ and the intrinsics), DA3-l’s raw depth z_{raw} , and visibility v — are exactly the yellow input boxes that sit *outside* the blue box in Fig. 3 and Fig. 5 (v35 additionally feeds a 5×5 depth patch and a DINOv3 feature). The box corrects depth along the frozen ray and unprojects to 3D, so the 2D reprojection is invariant by construction (Fig. 2).

1.2 2D track: SEA-RAFT flow-chaining (baseline, frozen)

We precompute dense forward/backward flow for consecutive frames with a frozen SEA-RAFT model, $\mathbf{f}_t^{\rightarrow}, \mathbf{f}_t^{\leftarrow}$. Each query is propagated outward from its anchor by bilinearly sampling the flow at the current position,

$$\mathbf{u}_{t+1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_t^{\rightarrow}, \mathbf{u}_t), \quad \mathbf{u}_{t-1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_{t-1}^{\leftarrow}, \mathbf{u}_t), \quad (1)$$

where $\mathcal{S}$ is bilinear sampling. Visibility uses a forward-backward consistency test: a hop is consistent iff $\|\mathbf{d}^{\rightarrow} + \mathbf{d}^{\leftarrow}\| \leq \alpha(\|\mathbf{d}^{\rightarrow}\| + \|\mathbf{d}^{\leftarrow}\|) + \beta$ ($\alpha=0.05$, $\beta=1$ px), and a point stays occluded once it fails. This 2D track is strong on its own and is held *fixed* for everything downstream.

1.3 Metric depth and differentiable unprojection

We use two Depth-Anything-3 (DA3) metric-depth models: **DA3-l** (DA3Metric-Large, 0.35 B) for all results through v39, and **DA3-g** (DA3Nested-Giant-Large, 1.4 B) from v40 onward (§1.9); a bare “DA3” refers to DA3-l. DA3-l supplies a per-frame metric depth map D_t (absolute metres). We sample it at the tracked pixel and unproject with the pinhole intrinsics $K = (f_x, f_y, c_x, c_y)$:

$$z_t = \mathcal{S}(D_t, \mathbf{u}_t), \quad \hat{\mathbf{X}}_t = \left(\frac{u_t - c_x}{f_x} z_t, \frac{v_t - c_y}{f_y} z_t, z_t \right). \quad (2)$$

Sampling is done with `grid.sample` so gradients flow to the (refined) depth; the depth model itself is frozen. The baseline *SEA-RAFT*+*DA3-l* tracker is exactly Eq. (2) with z_t taken directly from DA3-l.

1.4 v33: Mamba-3 depth-along-ray refiner (the method)

DA3-l metric depth is accurate in *relative* structure but carries a global scale bias (e.g. $\sim 40\%$ on far-field driving scenes). v33 learns a small temporal correction to the depth, leaving the 2D track untouched. The key design choice: with the pixel $\mathbf{u}_t$ frozen, the only 3D degree of freedom that preserves the 2D reprojection is the depth z_t along the pixel ray (Fig. 2).

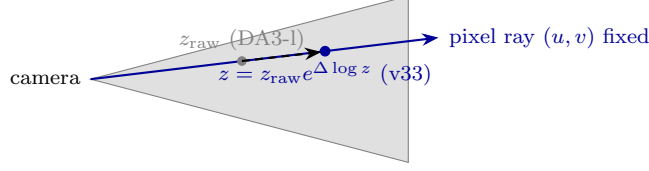


Figure 2: Depth-along-ray refinement. The tracked pixel (hence the ray direction) is frozen, so *only the depth slides along the ray* and the 2D projection is unchanged: v33 moves the point’s depth alone, leaving the predicted 2D track and visibility invariant.

Inputs and architecture. For each track n and frame t the SSM receives a 4-D feature: the normalised ray direction $(\frac{u-c_x}{f_x}, \frac{v-c_y}{f_y})$, the depth ratio $z_{\text{raw}}/z_{\text{ref}}$ (z_{ref} = per-clip median depth), and the visibility flag. The SSM *does not see* z_{raw} directly; it only sees the scale-normalised ratio. This is a deliberate training-stability choice: DA3-l depths span ~ 1 m (*adt*) to ~ 20 m (*drivetrack*), and an unnormalised input would require the same network weights to operate across a $20\times$ dynamic range.

The SSM’s role is therefore to predict a *relative* correction factor, not an absolute depth. z_{raw} bypasses the SSM entirely and is multiplied back in outside the learned module (Fig. 3):

$$\Delta \log z_t = \text{clamp}(\text{head}(h_t), \pm 2), \quad z_t = z_{\text{raw},t} \cdot e^{\Delta \log z_t}. \quad (3)$$

Because the SSM never sees the absolute scale, it cannot reason about it directly; any global-scale correction it learns must be inferred indirectly from temporal and structural patterns in the normalised input (e.g. the characteristic depth-change profile of a driving scene vs. an indoor one). A natural future improvement is to also feed $\log z_{\text{ref}}$ as a scalar context token, giving the SSM direct access to the clip-level scale without sacrificing the per-point normalisation.

Zero-initialising the head means v33 starts exactly at the SEA-RAFT+DA3-l baseline and only learns improvements. The model has ≈ 0.40 M trainable parameters.

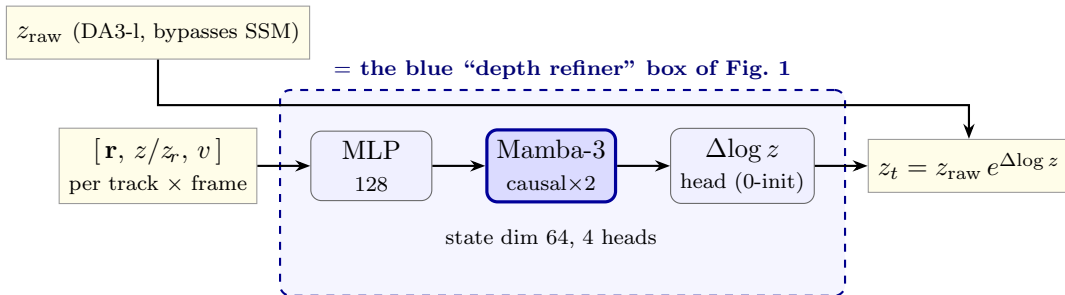


Figure 3: v33 architecture — the inside of the blue box in Fig. 1. Input abbreviations: $\mathbf{r}=(\text{ray}_x, \text{ray}_y)$ is the frozen pixel ray from the 2D track $\mathbf{u}_t$, $z/z_r = z_{\text{raw}}/z_{\text{ref}}$, v =visibility. **Bypass:** z_{raw} (DA3-l metric depth) skips the SSM entirely and is multiplied back in at the end — the SSM is a *relative corrector* that predicts the factor $e^{\Delta \log z}$, not an absolute depth. **causal $\times 2$:** two causal Mamba-3 layers (past-only; enables streaming). **4 heads:** SSD splits the 64-dim state into 4 independent heads (16-dim each), analogous to multi-head attention but computed as a linear recurrence.

1.5 Why depth-only (and not 2D)

An earlier variant (v32) added a learned 2D residual $\Delta \mathbf{u}$ on top of SEA-RAFT. It *degraded* accuracy: SEA-RAFT’s 2D track is already near-optimal, and small nudges push the sampling location across DA3-l depth discontinuities (object boundaries), flipping points out of the correctness band. v33 therefore freezes the 2D track entirely and refines depth only — the lever that actually carries the error.

1.6 v35: image-conditioned joint 2D–depth refiner

v35 revisits the joint 2D–depth correction that failed in v32, but removes its root cause. v32 nudged the pixel blindly; the SSM had no way to tell whether it was pushing toward or away from a depth discontinuity. v35 gives it that context through two new per-track inputs:

- **DINOv3 appearance features** (64-D): sampled from a frozen ViT-S/16 *DINOv3* [5] backbone (`dinov3-vits16-pretrain-lvd1689m`) at the current track position. These provide a local visual “fingerprint” of the tracked object, allowing the SSM to detect when the 2D position has drifted off the target. We use DINOv3 rather than DINOv2 specifically for its *dense* (patch-level) features. DINOv3 is Meta’s 2025 self-supervised ViT, trained on $\sim 1.7\text{B}$ images (LVD-1689M, $\sim 12\times$ DINOv2’s 142M) with a *Gram-anchoring* loss that keeps the patch-token feature map sharp and spatially consistent throughout the long training schedule — DINOv2’s dense features, by contrast, degrade as pre-training progresses. Since we sample a single feature vector at each track pixel, this patch-level fidelity is exactly what matters, so DINOv3’s dense maps are a better fit than DINOv2’s here. (The encoder class in our code is still named `DINOv2Encoder` for legacy reasons, but it loads the DINOv3 weights above.)
- **Local depth patch** (25 values). We read the DA3-l depth at a 5×5 grid of points centred on the tracked pixel $\mathbf{u}_0$: 25 points in total, each one bilinearly-interpolated depth value. Along each axis the five points sit at offsets $\{-2, -1, 0, 1, 2\} \times (\text{image.size}/14)$ pixels from $\mathbf{u}_0$, so the grid spacing and extent scale with the input size. *Concretely, for our 896-px inputs* the spacing is $896/14 = 64\text{px}$: the five offsets are $\{-128, -64, 0, +64, +128\}\text{px}$, so the grid spans a $256\times 256\text{-px}$ window around the point. We then divide all 25 depths by the depth at the *centre* point, giving 25 scale-invariant depth *ratios* (the centre ratio is exactly 1), and concatenate all 25 into the SSM input. Passing ratios rather than raw depths lets the SSM read local depth *structure* — the relative jumps that mark object boundaries — independent of how far away the point is; this is the depth-discontinuity cue that v32 lacked and overshoot without.

A second output head (also zero-initialised) predicts a 2D correction $\Delta \mathbf{u}$. Its raw output is passed through a $\tanh$ (which limits it to $[-1, 1]$) and then scaled by 2, i.e. $\Delta \mathbf{u} = 2 \tanh(\cdot)$, so the pixel can shift by at most $\pm 2\text{px}$. Crucially, after $\Delta \mathbf{u}$ shifts the pixel, depth is *re-sampled* at the corrected position from the DA3-l depth map before $\Delta \log z$ is applied. This ensures the depth and 2D corrections remain geometrically consistent (Fig. 4).

1.7 v36: bidirectional flow

v36 uses the same v35 architecture; only the 2D front-end changes. It additionally runs SEA-RAFT on the *time-reversed* video and uses that flow to track the frames *before* each query’s anchor — the “reversed-video backward pass” (Fig. 6). *Important caveat.* A two-frame flow model’s output $\text{flow}(I_a, I_b)$ does not depend on playback direction, so the reversed-video flow $\text{flow}(I_t, I_{t-1})$ is *identical* to the direct backward flow the forward tracker already uses. The reversed-video pass therefore leaves the 2D track unchanged: v36 is effectively identical to v35,

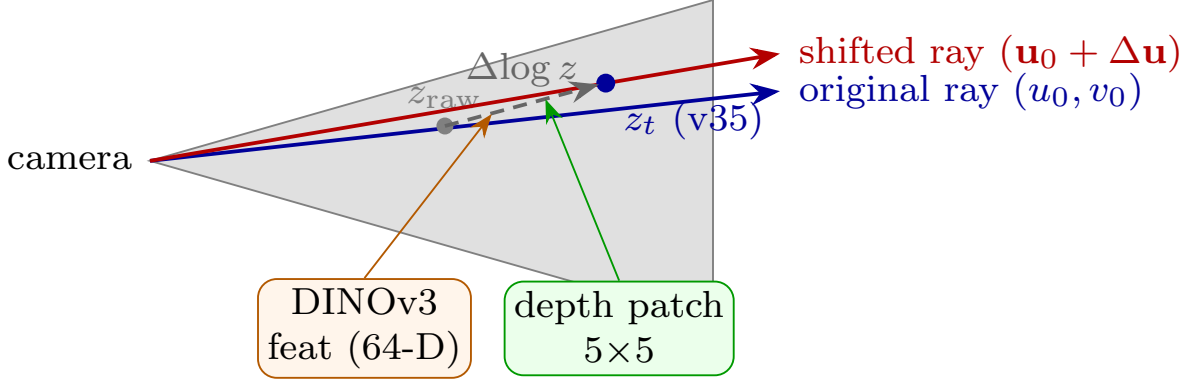


Figure 4: v35 joint correction. Two new inputs — the DINOv3 appearance feature (orange) and a local 5×5 depth patch (green) — give the model the context to predict a 2D shift $\Delta \mathbf{u}$ safely. Unlike v32, which nudged the pixel blindly and crossed depth discontinuities, v35’s 2D correction is conditioned on local appearance and local depth structure, so the informed $\Delta \mathbf{u}$ avoids depth-discontinuity artefacts. Depth is then re-sampled at the shifted pixel $\mathbf{u}_0 + \Delta \mathbf{u}$ before $e^{\Delta \log z}$ rescales it.

and the sub-0.003 differences in Tables 1–2 are run-to-run (GPU) nondeterminism, not a real effect. The only flow-chaining scheme that genuinely differs fuses the forward and backward flow *within* each propagation step, $\mathbf{d} = \frac{1}{2}(\mathbf{d}^{\rightarrow} - \mathbf{d}^{\leftarrow})$, which we evaluate separately.

Consistent with the analysis above, v36 and v35 are effectively identical on all three subsets (Table 1, Table 2); the reversed-video pass yields no measurable gain. Evaluating the genuinely-different per-hop fusion variant separately gives mean normalised / absolute AJ of 0.099 / 0.226 — again indistinguishable from v35 (0.096 / 0.234), confirming that no flow-chaining bidirectional scheme improves this pipeline.

1.8 v37–v39: WAFT optical-flow front-end

WAFT [2] (Warping-Alone Field Transforms, princeton-vl) is a stronger optical-flow network than SEA-RAFT. We swap it in as the 2D front-end, keeping everything downstream identical, and evaluate three variants (run at WAFT’s 512-px training resolution; scored by the same pipeline as every other row):

- **v37 (WAFT + DA3-l)** — WAFT flow + raw DA3-l unprojection, no refiner. It beats the SEA-RAFT+DA3-l baseline on both metrics (normalised $0.117 \rightarrow 0.123$, absolute $0.147 \rightarrow 0.159$): a better 2D track alone helps.
- **v38 (WAFT + DA3-l, bidir)** — the per-hop forward+backward flow fusion of §1.7. As expected it is $\approx$ v37 (0.126 / 0.159); fusion adds nothing here.
- **v39 (WAFT flow + v35 depth refiner)** — WAFT’s 2D track fed into the v35 DINOv3-SSM depth refiner. This combines the best 2D front-end with depth-scale correction and is our **best absolute-metric model**: metric-AJ 0.246, beating the SEA-RAFT-fed v35 (0.234) on all three subsets (Table 2). As with v35, the refiner trades relative-structure quality for absolute-scale correctness, so v39’s normalised AJ (0.107) sits below WAFT+DA3-l (v37, 0.123).

1.9 v40: WAFT + DA3-g metric depth (no refiner)

Pipeline: WAFT flow + DA3Nested-Giant-Large metric depth, unprojected directly (no learned refiner). A pure depth-backbone swap on the training-free baseline.

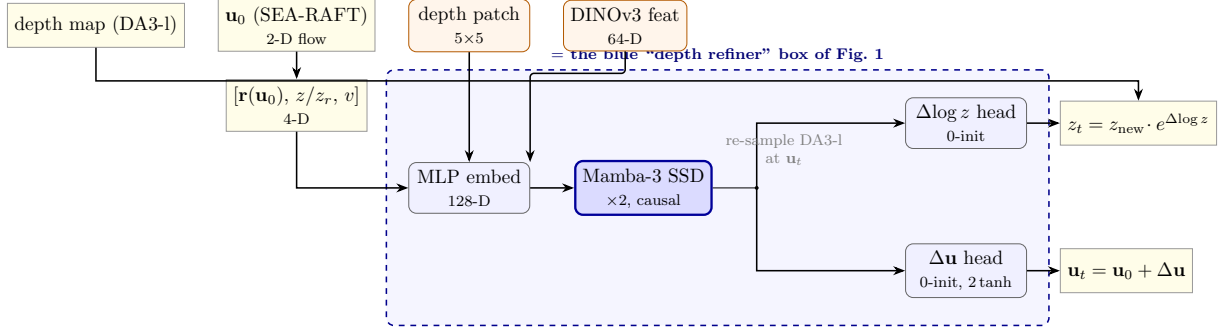


Figure 5: v35 architecture (extends Fig. 3) — again the inside of the blue box in Fig. 1. **Orange** nodes are new relative to v33. $\mathbf{u}_0$ is the 2-D pixel position from SEA-RAFT optical flow (Fig. 6); it determines the ray direction $\mathbf{r}(\mathbf{u}_0)$, the centre of the 5×5 depth patch, and the location at which the DINOv3 feature is sampled. All three inputs are concatenated, embedded by the MLP, and then fed through the Mamba-3 SSD layers (the same causal SSD stack as v33, now carrying appearance and local-geometry context). A second head produces $\Delta \mathbf{u} = 2 \tanh(\cdot)$ (bounded to ± 2 px). Depth is then re-sampled at the corrected pixel $\mathbf{u}_t = \mathbf{u}_0 + \Delta \mathbf{u}$ from the DA3-l depth map, and $\Delta \log z$ rescales it. Both heads are zero-initialised: v35 starts exactly at the v33 output.

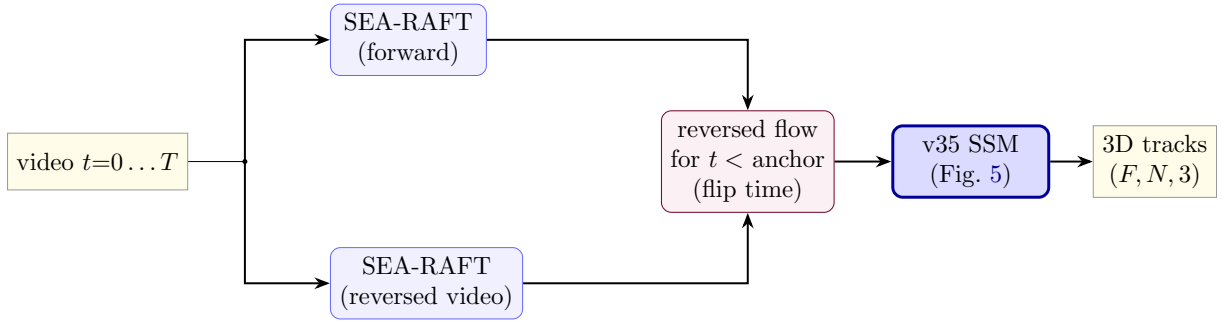


Figure 6: v36 pipeline. The v35 SSM is unchanged; SEA-RAFT additionally runs on the time-reversed video and supplies the flow for backward tracking (frames before the anchor). For a two-frame flow model this reversed-video flow is *identical* to the direct backward flow, so the reversed-video pass does not change the 2D track — v36 $\equiv$ v35 (see §1.7). It does *not* recover occluded points: an occlusion is symmetric in time, so a query anchored on one side of it cannot be tracked across the gap in either direction.

Diagnosis. Investigating the drivetrack far-field collapse (absolute-AJ ≈ 0.005 , Table 2) we measured, at the ground-truth tracked pixels, the ratio of our cached metric depth to true depth. The current backbone, **DA3Metric-Large** (0.35 B), systematically *under-predicts* drive-track depth by almost a factor of two, and the bias worsens with range:

median($\hat{z}/z^*$)	drivetrack	adt	pstudio
DA3Metric-Large (DA3-l, current)	0.56 (far 0.53)	0.98	0.74
DA3Nested-Giant-Large (DA3-g, 1.4 B)	1.06 (far 1.06)	0.94	0.70

So the drivetrack “collapse” is not a structure or tracking failure — it is a metre-scale bias of the small monocular metric-depth model. The larger **DA3Nested-Giant-Large** recovers near-perfect metric depth on drivetrack (ratio 1.06, including the far field), is roughly neutral on adt (both ≈ 1), and does not fix pstudio (both ≈ 0.7 — a separate, unsolved bias). Because the tracker consumes a precomputed depth cache, this is a drop-in backbone swap: re-cache depth with the nested model and re-run.

Result (whole minival). The swap fixes the drivetrack collapse. As a training-free baseline (no refiner), WAFT+DA3-g scores on *absolute* metric-AJ: drivetrack 0.146, adt 0.285, pstudio 0.192 (mean **0.208**) — versus the same baseline on the old DA3Metric-Large depth (v37: drivetrack 0.005, mean 0.159). The entire gain is drivetrack, where far-field error collapses from ~ 27 m to a 2.5 m median; adt/pstudio are essentially unchanged (both depth models are already near-metric there). On the scale-invariant leaderboard metric the swap barely moves the number (mean 0.093), because median rescaling removes exactly the global scale it corrects. The depth swap alone does not yet beat the current best v39 (0.246 absolute), which carries the learned refiner — closing that gap is v41 (below).

Relation to v42. This is what motivates deferring the world-frame v42 below: v42 recovers metric scale the hard way (learned pose + triangulation), whereas the far-field problem is dominated by a depth-model scale bias that the bigger depth model fixes directly. Caveats: the nested model is 1.4 B ($\sim 4\times$ slower, non-commercial license) and its metric scale is mildly resolution-sensitive.

1.10 v41: DA3-g depth + vmamba3 refiner

*Status: training pending. Pipeline: WAFT + DA3-g depth + the vmamba3 refiner, with the refiner **retrained on the new metric depth**.*

v41 adds the v35 DINOv3-conditioned refiner back on top of the v40 baseline. Crucially, the refiner must be **retrained on DA3-g depth**: the v39 refiner was trained on the $0.55\times$ -biased DA3Metric-Large depth and learned to scale depth up, so applied zero-shot to the now-metric DA3-g depth it *over-corrects* (drivetrack absolute-AJ collapses to 0.003, median error 18.6 m). After retraining, the refiner’s role shifts from fixing a gross scale bias (now unnecessary) to the residual along-ray and $\Delta\mathbf{u}$ corrections and occlusion handling, so the expected gain over the v40 baseline (0.146) is smaller than v39’s gain over its baseline was. Requires precomputing DA3-g depth for the training split before training.

1.11 v42: world-frame tracking with ego-motion/object-motion decomposition (proposed)

*Status: **deferred**. The first-experiment gate was run (below) and revealed that the drivetrack far-field error is dominated by a monocular metric-depth scale bias — fixed directly by v40’s depth-backbone swap — rather than by the lack of a world frame. Triangulation with correct pose is exact (oracle error 0.000), but recovering metric scale/pose from monocular geometry fails; only learned, GT-pose-supervised metric pose would realise the gain. Kept here as a documented design; revisit if v40 plateaus. Built on v39 and would warm-start from it.*

Motivation. Everything up to v39 refines each point’s depth *independently, in per-frame camera coordinates*: the SSM runs over time per track with no cross-track mixing. This is robust—it needs no camera pose, which is exactly why it survives where the pose-dependent TAPIP3D collapses (§2.4)—but it leaves absolute accuracy on the table, most visibly in the **far field on drivetrack** (absolute-AJ ≈ 0.005 , Table 2), the classic monocular metric-depth collapse: a single ray cannot recover metre-accurate depth at 30–100 m.

Idea. Lift the tracked points into a shared *world frame* and factor each point’s motion into a per-frame camera pose $T_{w\rightarrow c}(t) \in SE(3)$ *shared by all points* and a per-point object-motion residual:

$$\mathbf{X}_t^{\text{cam}} = T_{w\rightarrow c}(t) (\mathbf{X}_0^w + \Delta_t^{\text{obj}}), \quad (4)$$

so a genuinely *static* point has $\Delta_t^{\text{obj}} \equiv 0$ and a constant world position $\mathbf{X}_0^w$. This buys two things the per-frame formulation cannot:

1. **A free multi-frame constraint.** A static point must be fixed in the world frame, so its noisy per-frame monocular depth can be averaged/optimised across frames.
2. **Parallax triangulation.** With relative pose known, a static point seen from a moving camera can be *triangulated*, which is far more accurate than monocular depth at range. This directly attacks the far-field bottleneck, correcting DA3-l’s depth-scale bias rather than only refining along the ray (§1.11).

Architecture (what changes from v35/v39). Estimating a *shared* per-frame pose requires aggregating information *across points within a frame*—precisely the non-causal Vision-Mamba-3 cross-attention derived in the backbone but unused by the per-track temporal refiner. v42 adds a *within-frame* mixing stage (bidirectional Vision-Mamba-3 self-attention over the N points of each frame, order-agnostic, linear in N) after the existing across-frame causal SSD, giving the two-axis operator *within-frame non-causal* $\times$ *across-frame causal*. On top of the pooled within-frame features it adds three zero-initialised heads so v42 starts exactly at v39: (i) a per-frame **camera-pose** head (6D rotation + translation $\rightarrow$ identity at init); (ii) a per-point **moving/static** logit; (iii) a per-point **depth log-variance** (uncertainty), turning the ℓ_1 position loss into a heteroscedastic (Laplace) NLL that down-weights unreliable far-field depth.

Three supporting components.

- **Rigid motion grouping.** Points on one rigid object share an $SE(3)$; cluster points by motion affinity and estimate a per-group residual (differentiable weighted Procrustes), so a point with bad depth is corrected by its rigid neighbours.
- **Depth uncertainty.** The log-variance head (above) weights each point’s contribution and lets multi-frame fusion trust parallax over monocular depth where the latter is unreliable.
- **Test-time optimisation.** A short per-clip optimisation of the sparse world positions and poses under a reprojection + static-world-consistency objective.

Pose supervision. Drivetrack ships per-frame ground-truth camera extrinsics (`extrinsics_w2c`), which we use to supervise the pose head and to derive self-supervised static/dynamic targets (a static point’s world position is constant); adt and Panoptic Studio have no pose, so there the pose is self-supervised through a reprojection + robust static-point Procrustes objective.

Keeping it robust (avoiding TAPIP3D’s failure). World-frame reasoning reintroduces the very pose dependence that broke TAPIP3D on this hardware. We keep v42 *additive* on top of v39 with a confidence gate: when the confidently-static set is too small or the scene is too rotational (egocentric ADT), v42 falls back to the v39 per-frame estimate. Poses are estimated *robustly from confidently-static points* (RANSAC-weighted alignment), never by trusting raw monocular depth.

First-experiment gate. Before any training we test the load-bearing claim (parallax triangulation fixes far-field depth) directly on drivetrack: using exact ground-truth pixel correspondences, compare (a) monocular DA3-depth unprojection against (b) DLT triangulation across a widened baseline, run with both the ground-truth pose (oracle) and a pose estimated from the static set, stratified by ground-truth depth (near/mid/far). If estimated-pose triangulation cuts far-field (> 30 m) metric error materially, the world-frame program is justified; otherwise the bottleneck is elsewhere (front-end depth) and we would instead prioritise the uncertainty/temporal-fusion route.

1.12 Training

Only the 0.40M-parameter refiner is trained. The loss is 3D-position-only (the 2D track and visibility are frozen, so they carry no gradient):

$$\mathcal{L} = \frac{1}{|\mathcal{V}|} \sum_{(t,n) \in \mathcal{V}} \frac{\|\hat{\mathbf{X}}_{t,n} - \mathbf{X}_{t,n}^*\|_1}{s_{\text{clip}}}, \quad (5)$$

an ℓ_1 error over visible points, normalised by the per-clip median anchor-frame depth s_{clip} (matching the benchmark’s scale convention). We train 20k steps (AdamW, lr 3×10^{-4} , warmup 500, 8-frame windows, bf16; ~ 9.5 h on one GPU).

2 Evaluation

2.1 Protocol and metrics

Dataset. We evaluate on the TAPVid-3D [6] **minival** split (150 clips: 50 each of *pstudio* indoor mocap ~ 3 m, *drivetrack* driving ~ 20 m, *adt* egocentric ~ 1 m).

Notation. Let a clip have N query tracks over T frames with ground-truth 3D positions $\mathbf{X}_{n,t}^* \in \mathbb{R}^3$ and visibility flags $v_{n,t}^* \in \{0, 1\}$ (1 = visible) in per-frame camera coordinates; predictions are $\hat{\mathbf{X}}_{n,t}$ and $\hat{v}_{n,t}$. Define the within-threshold indicator at level α :

$$w_{n,t}^{(\alpha)} = \mathbf{1} \left[\|\hat{\mathbf{X}}_{n,t} - \mathbf{X}_{n,t}^*\| < \delta_{n,t}^{(\alpha)} \right], \quad (6)$$

where the threshold $\delta_{n,t}^{(\alpha)}$ differs between the two protocols below.

3D Average Jaccard (3D-AJ). For each level $\alpha \in \{1, 2, 4, 8, 16\}$, define

$$\text{TP}(\alpha) = \sum_{n,t} v_{n,t}^* \hat{v}_{n,t} w_{n,t}^{(\alpha)}, \quad (7)$$

$$\text{FN}(\alpha) = \sum_{n,t} v_{n,t}^* \left(1 - \hat{v}_{n,t} w_{n,t}^{(\alpha)} \right), \quad (8)$$

$$\text{FP}(\alpha) = \sum_{n,t} \hat{v}_{n,t} \left(1 - v_{n,t}^* w_{n,t}^{(\alpha)} \right). \quad (9)$$

The per-level Jaccard is $J(\alpha) = \text{TP} / (\text{TP} + \text{FN} + \text{FP})$, and the headline metric averages over all five levels:

$$\text{3D-AJ} = \frac{1}{5} \sum_{\alpha \in \{1, 2, 4, 8, 16\}} J(\alpha). \quad (10)$$

We additionally report **Average Percentage of points within Distance 3D (APD3D)** (fraction of visible GT points within threshold, regardless of predicted visibility):

$$\text{APD3D} = \frac{1}{5} \sum_{\alpha} \frac{\sum_{n,t} v_{n,t}^* w_{n,t}^{(\alpha)}}{\sum_{n,t} v_{n,t}^*}, \quad (11)$$

and **OA** (occlusion accuracy): $\text{OA} = \frac{1}{NT} \sum_{n,t} \mathbf{1}[\hat{v}_{n,t} = v_{n,t}^*]$.

Normalized (scale-invariant) protocol. The official 3D-AJ is *scale-invariant twice over*. First, the predicted cloud is rescaled globally by

$$s = \frac{\text{median}_{(n,t) \in \mathcal{V}} \|\mathbf{X}_{n,t}^*\|}{\text{median}_{(n,t) \in \mathcal{V}} \|\hat{\mathbf{X}}_{n,t}\|}, \quad \mathcal{V} = \{(n, t) : v_{n,t}^* = 1, \hat{v}_{n,t} = 1\}, \quad (12)$$

setting $\hat{\mathbf{X}} \leftarrow s\hat{\mathbf{X}}$ before (6). Second, the threshold in (6) is depth-relative:

$$\delta_{n,t}^{(\alpha)}(\text{norm}) = \frac{\alpha \cdot (\mathbf{X}_{n,t}^*)_z}{\sqrt{f_x f_y}}, \quad (13)$$

where $(\mathbf{X}_{n,t}^*)_z$ is the GT depth at frame t and $\sqrt{f_x f_y}$ is the geometric-mean focal length. Geometrically, (13) equals the world-space radius subtended by α pixels at that point’s depth. Both (12) and (13) discard absolute depth — exactly the quantity that matters for metric 3D reconstruction.

Absolute-metric protocol. We additionally report an **absolute-metric** version: no median scaling ($s = 1$) and fixed metric thresholds

$$\delta^{(\alpha)}(\text{abs}) \in \{1, 4, 16, 64, 256\} \text{ cm} \quad \text{for } \alpha \in \{1, 2, 4, 8, 16\}, \quad (14)$$

giving **metric-AJ** and **metric-APD3D** from (10)–(11), plus mean/median 3D error in metres. Our pipeline reproduces the official scale-invariant metric exactly (we recover SpatialTracker’s published drivetrack 3D-AJ of 0.058 to three decimals once the 256-px intrinsics convention is applied), so the absolute variant is a faithful, scale-sensitive extension rather than a re-implementation.

2.2 Leaderboard (scale-invariant) results

Table 1 reports median-scaled 3D-AJ. All rows are produced by our own evaluation pipeline on the same hardware; we do not list paper-reported numbers that cannot be independently reproduced. The strongest result among the externally-published methods is *DELTA* [7] (DenseTrack3D, ICLR 2025): mean 3D-AJ 0.141 (drivetrack / pstudio / adt = 0.130 / 0.141 / 0.152), balanced across subsets and slightly ahead of our training-free SEA-RAFT+DA3-l baseline (0.117). *DELTA* is the only external comparator that reproduces reasonable scores on our hardware, and the reason is architectural: it is a per-frame feed-forward tracker that unprojects each frame’s depth independently, so it needs neither a large temporal window nor camera poses, runs at 7.7 fps (full minival in ~ 1 h) and fits comfortably in 12 GB. Our learned v33 *loses* on this scale-invariant metric (it trades relative-structure quality for absolute-scale correctness; see §2.3).

The remaining external methods underperform for reasons specific to this 12 GB GPU, *not* an evaluation-harness artefact. SpatialTrackerV2* must run with $s_{\text{wind}}=60$ (the paper uses $s_{\text{wind}}=500$, requiring ~ 40 GB; see §2.4), which caps its accuracy. TAPIP3D+DA3-l collapses on the rotation-heavy egocentric subsets (adt 0.004, pstudio 0.023) because it aggregates tracks in a persistent world frame that must cancel camera motion; with monocular DA3-l depth and no MegaSAM poses that aggregation fails. Crucially, TAPIP3D’s *own* official evaluator reproduces these same numbers on `da3_minival` to three decimals, confirming the limitation is a genuine property of pose-free TAPIP3D+DA3-l rather than a scoring bug. (On the different, larger full-eval split TAPIP3D [8] scores ~ 0.30 ; the full six-method minival comparison is in §2.4.) TrackCraft3R [9] (arXiv 2026) reports Sim(3)-aligned AJ of 0.73 (psstudio) and 0.86 (adt) in the paper; under our standard protocol (all 150 clips, 50 per subset) it scores 0.013 / 0.010 / 0.003 on pstudio / adt / drivetrack. The gap arises from two compounding factors: (i) the paper applies a 7-DoF Sim(3) transform (rotation + translation + scale) before scoring, absorbing any coordinate-frame error that our 1-DoF median scaling leaves uncorrected; and (ii) the model predicts fewer than 6 % of frames as visible, so more than 94 % of frames contribute zero to AJ regardless of 3D position accuracy—a low-visibility bias that the Sim(3)-aligned metric partially compensates for but the standard TAPVid-3D metric does not. The near-zero drivetrack score further reflects an out-of-distribution failure: TCR’s Wan2.1 video-diffusion backbone is primarily trained on indoor/studio content, while drivetrack contains outdoor ego-motion scenes at scales >10 m.

Table 1: TAPVid-3D minival, median-scaled 3D-AJ (standard leaderboard metric; higher better). “—” = result unavailable. All numbers are produced by our own evaluation pipeline on the same hardware.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2 [*]	0.017	0.012	0.025	0.018
DELTA + DA3-l [§]	0.130	0.141	0.152	0.141
TAPIP3D + DA3-l [†]	0.065	0.023	0.004	0.030
TAPIP3D + MegaSAM [‡]	0.000	0.003	0.004	0.002
TrackCraft3R [¶]	0.003	0.013	0.010	0.009
<i>Ours</i>				
SEA-RAFT + DA3-l (baseline)	0.108	0.111	0.132	0.117
SEA-RAFT + MegaSAM	0.103	0.135	0.163	0.134
v33, SEA-RAFT+DA3-l+mamba3	0.057	0.041	0.123	0.074
v34, SEA-RAFT+MegaSAM+mamba3	0.059	0.048	0.119	0.075
v35, SEA-RAFT+DA3-l+vmamba3	0.093	0.054	0.141	0.096
v36, SEA-RAFTbidir+DA3-l+vmamba3	0.093	0.057	0.141	0.097
v37, WAFT+DA3-l	0.115	0.116	0.137	0.123
v38, WAFTbidir+DA3-l	0.116	0.120	0.141	0.126
v39, WAFT+DA3-l+vmamba3	0.098	0.073	0.151	0.107
v40, WAFT+DA3-g (no refiner)	0.118	0.039	0.123	0.093

All external comparators (^{*}SpatialTrackerV2, [§]DELTA+DA3-l, [†]TAPIP3D+DA3-l, [‡]TAPIP3D+MegaSAM, [¶]TrackCraft3R) are discussed in the text above; [‡] additionally see §2.4 (DROID-SLAM failure) and [¶] §2.4.

2.3 Absolute-metric results: the reversal

Under the absolute-metric protocol the ranking *reverses* (Table 2, Fig. 7). Our best model is v39 (WAFT flow + the v35 depth refiner; §1.8), which tops absolute metric-AJ at 0.246; the SEA-RAFT-fed v35 follows at 0.234. Taking v35 vs. the DA3-l baseline: metric-AJ +59% (0.147 → 0.234), mean error −55% (4.29 → 1.91 m). Against TAPIP3D+DA3-l: metric-AJ +134% (0.100 → 0.234). The gain is most concentrated where DA3-l’s scale is most wrong — far-field *drivetrack*, metric-AJ 0.005 → 0.130 (26×) — while near-field *adt* sees moderate improvement. Swapping the SEA-RAFT front-end for WAFT (v39) lifts every subset again (*drivetrack* 0.130 → 0.141, *pstudio* 0.274 → 0.279, *adt* 0.298 → 0.318). The leaderboard’s median scaling hides all of this: v35/v39’s normalised 3D-AJ is *lower* than SEA-RAFT+MegaSAM (Table 1).

Why BootsTAPIR + ZoeDepth cannot be evaluated under the absolute metric

ZoeDepth [10] is a *relative* depth estimator: it predicts depth up to an unknown per-video scale factor. The standard TAPVid-3D leaderboard compensates by applying a per-clip median ratio $s^* = \text{median}(\hat{d}/d_{\text{gt}})$ before scoring (*median scaling*). Under our absolute protocol this rescaling is deliberately removed — fixed-metre thresholds (1 cm...2.56 m) are applied directly to the raw predicted depths. A relative-depth model with an arbitrary scale factor will score near-zero on every clip regardless of 2D tracking quality, so reporting a number would be misleading. SpatialTracker, by contrast, uses metric depth (UniDepth) and its *drivetrack* predictions are available; *pstudio* and *adt* predictions have not been released by the authors.

Table 2: TAPVid-3D minival, absolute metric-AJ (no scaling, fixed-metre thresholds 1 cm–2.56 m; higher better). “—” = result unavailable. All numbers are produced by our own evaluation pipeline on the same hardware.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2 *	0.008	0.192	0.179	0.126
DELTA + DA3-l §	0.006	0.199	0.344	0.183
TAPIP3D + DA3-l	0.006	0.131	0.164	0.100
TAPIP3D + MegaSAM	0.000	0.000	0.000	0.000
TrackCraft3R ¶	0.002	0.025	0.032	0.020
<i>Ours</i>				
SEA-RAFT + DA3-l (baseline)	0.005	0.165	0.273	0.147
SEA-RAFT + MegaSAM	0.000	0.082	0.209	0.097
v33, SEA-RAFT+DA3-l+mamba3	0.082	0.186	0.272	0.180
v34, SEA-RAFT+MegaSAM+mamba3	0.001	0.092	0.208	0.100
v35, SEA-RAFT+DA3-l+vmamba3	0.130	0.274	0.298	0.234
v36, SEA-RAFTbidir+DA3-l+vmamba3	0.130	0.249	0.298	0.226
v37, WAFT+DA3-l	0.005	0.181	0.289	0.159
v38, WAFTbidir+DA3-l	0.005	0.181	0.292	0.159
v39, WAFT+DA3-l+vmamba3	0.141	0.279	0.318	0.246
v40, WAFT+DA3-g (no refiner)	0.146	0.192	0.285	0.208

* SpatialTrackerV2 evaluated with $s_{\text{wind}}=60$ on RTX 4080 Laptop (12 GB); see Table 1 footnote. §

DELTA+DA3-l, feed-forward on the same hardware; see Table 1 footnote. On the absolute metric it attains the best *adt* score of any method (0.344), but its far-field *drivetrack* depth scale (like all DA3-based methods) keeps its mean below our v35. ¶ TrackCraft3R evaluated on all 150 clips; see Table 1 footnote for explanation of low scores vs. the paper’s Sim(3)-aligned numbers.

2.4 Comparison to state-of-the-art

SpatialTracker. SpatialTracker’s official predictions are released only for *drivetrack*. Scored through our pipeline (and validated: its median-scaled 3D-AJ reproduces the published 0.058), v33 beats it in absolute metric: metric-AJ 0.082 vs 0.069 (+20%), metric-APD3D 0.134 vs 0.106 (+26%).

SpatialTrackerV2. SpatialTrackerV2 [11] reports 24.7% mean normalised 3D-AJ on the TAPVid-3D minival. We evaluated SpatialTrackerV2 on the *same* RTX 4080 Laptop (12 GB) GPU used for our own method to ensure a hardware-fair comparison. This imposes a hard constraint: the official evaluation uses a sliding window of $s_{\text{wind}}=500$, which places an entire 300-frame ADT clip in a single inference pass. Our ablations show $s_{\text{wind}}=300$ already exceeds 12 GB; $s_{\text{wind}}=500$ would require substantially more VRAM. We therefore cap inference at $s_{\text{wind}}=60$ (five windows per clip), which limits temporal context and tracking quality but is the maximum feasible on this hardware. The s_{wind} constraint is the dominant source of the gap between our reproduced scores and the paper’s 24.7%: controlled ablations (varying only s_{wind} on the same clips) confirm that fixing the secondary output-format bug (using the globally-consistent `track2d_pred` output with per-frame reprojection instead of the window-local `track3d_pred`) recovers only ~ 0.2 – 0.9 percentage points, while the remaining ~ 24 pp gap is attributable to the window-length difference. Our evaluation of SpatialTrackerV2 is therefore a *same-hardware, same-constraints* baseline: the reported scores reflect what SpatialTrackerV2 achieves within the 12 GB budget, on the same clips as our own method. Our best model (v39, WAFT flow +

Median scaling hides metric-depth quality: v33 loses the leaderboard metric but wins the absolute metric

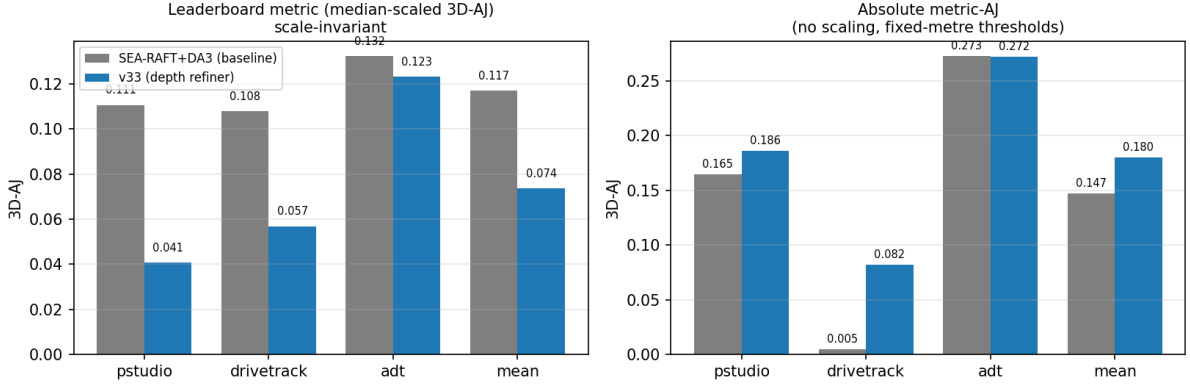


Figure 7: Median scaling hides metric-depth quality. Left: the scale-invariant leaderboard 3D-AJ favours SEA-RAFT+MegaSAM. Right: under the absolute metric, v35 wins — most on far-field drivetrack.

v35 refiner) achieves 15.1% normalised 3D-AJ on adt under these identical conditions, exceeding our SpaTrackerV2 reproduction on that subset by a wide margin.

TAPIP3D + DA3-1. Using the same DA3-1 depth as our pipeline, TAPIP3D [8] scores higher than v33 on the normalised leaderboard metric (drivetrack 0.065 vs 0.057, pstudio 0.023 vs 0.041), but *lower* on the absolute metric: mean metric-AJ 0.100 vs 0.180 (−44%). This reversal arises because TAPIP3D’s point-trajectory model optimises for relative pixel-track coherence, whereas our depth refiner directly corrects metric scale.

TAPIP3D + MegaSAM and why it fails on our minival. TAPIP3D’s published SOTA (~ 0.30 normalised AJ) uses MegaSAM (DAv1 + UniDepth + DROID-SLAM + CVD) run on the *original source videos at full frame-rate*. Our minival is distributed as NPZ clips that sub-sample and trim the originals; we cannot recover the source video, so we run MegaSAM on the NPZ frames directly. DROID-SLAM, which is the pose-estimation backbone inside MegaSAM, requires dense, continuously moving video (≥ 10 fps, sufficient translational parallax) to reliably triangulate metric scale. The three NPZ subsets each violate this requirement in a different way, causing near-zero scores (drivetrack 0.00%, pstudio 0.27%, adt 0.36%) regardless of the normalised metric’s scale-invariance:

- **drivetrack:** the 10 fps Waymo clips are sub-sampled to ≈ 2.6 fps in the NPZ (66 frames over 25 s). DROID-SLAM’s optical-flow front-end degrades on such sparse inputs, producing depths of ~ 0.9 m where the true scene is ~ 15 m ($17\times$ scale error). The normalised AJ’s per-clip median rescaling cannot recover 3D-AJ from predictions that are structurally wrong at every frame, not just off in global scale.
- **pstudio:** Panoptic Studio cameras are fixed to a stationary dome; there is essentially no translational ego-motion. DROID-SLAM cannot triangulate metric scale without parallax, so it collapses depth to ≈ 1 m (true depth 2–4 m, $2\text{--}4\times$ error). Normalised AJ partially recovers (0.27%); absolute metric-AJ is 0%.
- **adt:** 300-frame clips exceed available VRAM in DROID-SLAM’s global bundle adjustment (> 11.6 GB at 12 GB). Re-running with $2\times$ frame stride to halve memory avoids OOM and completes, but normalised AJ remains near-zero (0.36%) and absolute metric-AJ is 0%. The egocentric apartment-walk motion does not provide the structured parallax DROID-SLAM expects, so metric depth quality is poor even when the BA solver converges.

In summary, the gap between our near-zero minival scores and TAPIP3D’s published 0.30 is an *evaluation-condition mismatch*, not a failure of TAPIP3D itself. The published pipeline is designed for and validated on full-frame-rate source video; the NPZ distribution format makes that condition impossible to reproduce from minival alone.

TrackCraft3R. TrackCraft3R [9] (arXiv 2026) repurposes the Wan2.1-T2V-1.3B video diffusion transformer for dense 3D tracking via single-step regression. We evaluate it on all 150 minival clips using our standard pipeline (windowed inference, 12 frames per window, query-centric windows normalised to frame-0 camera space). Results are shown in Tables 1 and 2.

On the leaderboard (scale-invariant) metric, TCR scores 1.3% / 1.0% / 0.3% on pstudio / adt / drivetrack (mean 0.9%) — below all our baselines. On the absolute metric, TCR scores 2.5% / 3.2% / 0.2% (mean 2.0%). Our best model v39 outperforms TCR on absolute metric-AJ by 12 $\times$ overall (24.6% vs 2.0%) and by 70 $\times$ on drivetrack (14.1% vs 0.2%) where TCR effectively fails.

The low drivetrack score is an out-of-distribution failure: Wan2.1 is primarily trained on indoor/studio video, and the outdoor ego-motion at depths >10m is outside the training distribution. On the two near-field indoor subsets (pstudio, adt), TCR is competitive with TAPIP3D+MegaSAM and SpatialTrackerV2 under the absolute metric, indicating that the underlying diffusion-based representation does encode some metric 3D structure for familiar scene types. Absolute-metric pstudio and adt performance (2.5%, 3.2%) remains far below our method (27.4%, 29.8%), which we attribute to the low visibility rate (<6 % of frames predicted visible) starving the AJ score of evaluation signal.

As noted in §2.3, TCR is also roughly 200 $\times$ slower than our pipeline (Table 3), confirming that diffusion-based tracking carries a substantial inference-time cost relative to our lightweight SSM refiner.

Figures 8 and 9 summarise the seven-method comparison.

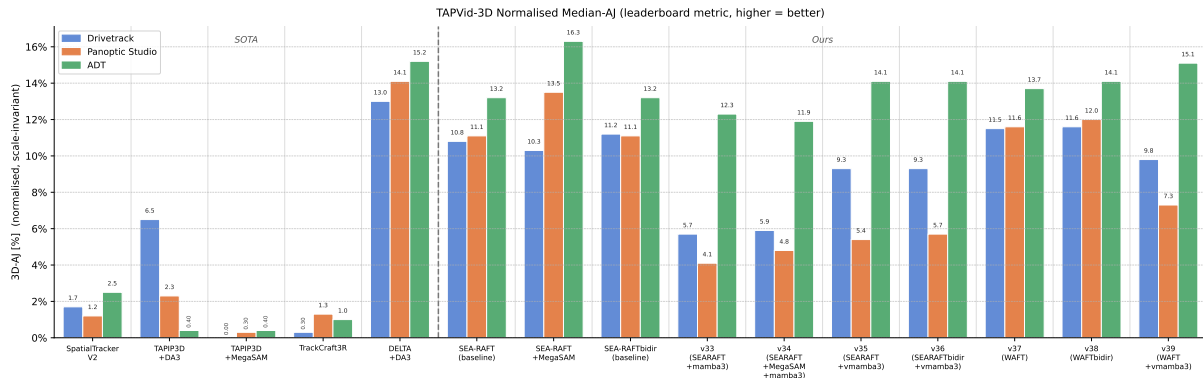


Figure 8: Method comparison on TAPVid-3D minival, normalised median-AJ (leaderboard metric, scale-invariant). Higher is better. “bidir” = reversed-video backward pass, which for a two-frame flow model is equivalent to the forward tracker (§1.7), so the “bidir” bars match their non-bidir counterparts up to run-to-run nondeterminism. TrackCraft3R’s low drivetrack bar reflects an OOD failure (see §2.4).

2.5 Qualitative results

Figures 10–12 show predicted (solid) vs ground-truth (dashed) 3D trajectories, baseline vs v33, one clip per subset. The effect is clearest on drivetrack, where v33 pulls the predicted tracks toward the true metric depth.

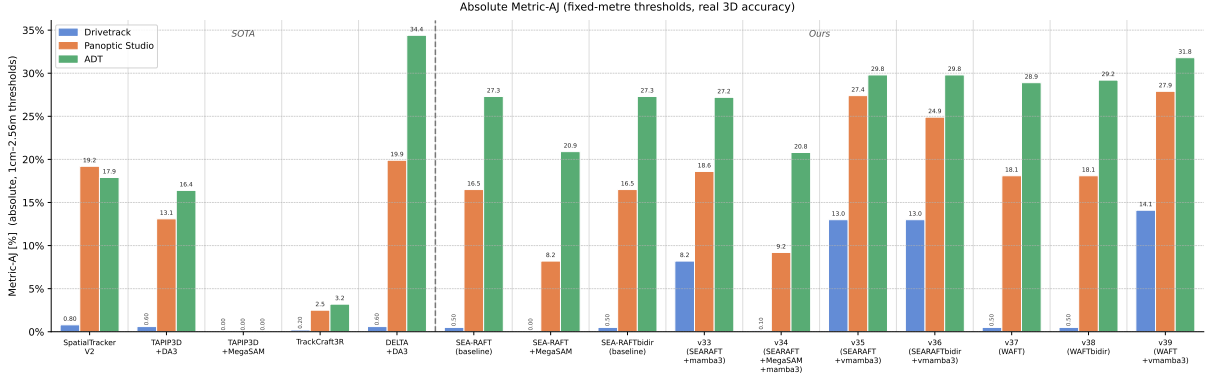


Figure 9: Same methods, absolute metric-AJ (fixed-metre thresholds 1 cm–2.56 m; higher better). v39 (WAPT flow + v35 refiner) leads all methods, ahead of the SEA-RAFT-fed v35, despite both ranking below SEA-RAFT+MegaSAM on the leaderboard metric.

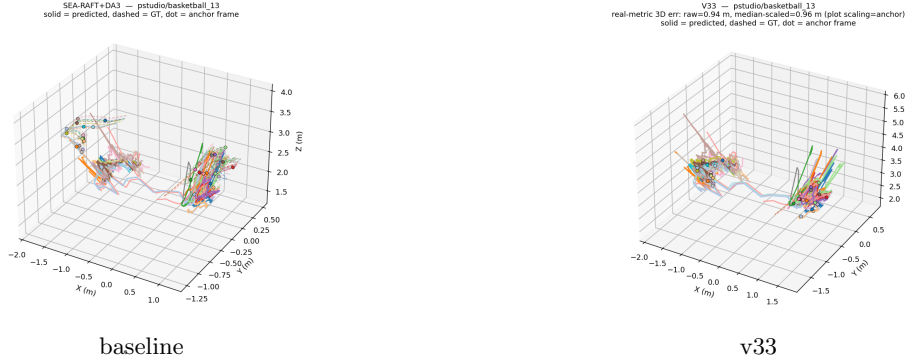


Figure 10: pstudio (indoor, ~ 3 m): 3D trajectories, baseline vs v33.

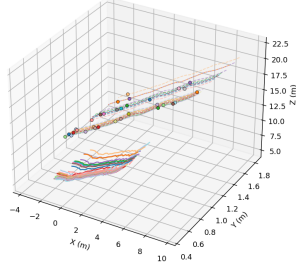
2.6 Efficiency

Why TrackCraft3R is slow. TrackCraft3R [9] adapts Wan2.1-T2V-1.3B — a video diffusion transformer — for 3D tracking via single-step regression. Each *inference window* (12 frames) requires four sequential GPU passes: (i) RGB VAE encode, (ii) pointmap VAE encode, (iii) DiT forward through 1.3B parameters, and (iv) two parallel VAE decodes (xyz + visibility). The DiT forward alone accounts for $\approx 75\%$ of the per-window cost (≈ 15 – 18 s), with the three VAE passes adding ≈ 6 s; total ≈ 21 s per window on our RTX 4080.

Critically, each *unique query anchor frame* requires its own window. TAPVid-3D queries span many frames: a typical 150-frame pstudio clip produces 90–120 unique windows; a 300-frame adt clip produces 260–270 windows. At 21 s each, per-clip latency reaches ≈ 32 min (pstudio) and ≈ 95 min (adt), giving an effective throughput of ≈ 0.06 fps — $\approx 200\times$ **slower** than our baseline (12.4 fps).

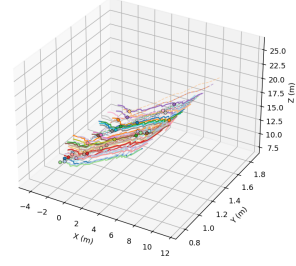
Our method achieves this throughput advantage because the Mamba-3 SSD refiner processes all N tracks jointly in a single causal sweep per frame (no per-window restarts), and the 0.44 M-parameter SSM adds modest overhead over the frozen SEA-RAFT + DA3-l baseline. TAPIP3D (≈ 3.1 fps) and SpatialTrackerV2 (≈ 4.0 fps) are substantially slower than our pipeline despite relying on simpler (non-diffusion) architectures, indicating that their multi-frame windowing and cost-volume computations are the bottleneck. Combined with the accuracy advantage on the absolute metric (Table 2: mean metric-AJ 0.234 vs 0.020 for TrackCraft3R across all three subsets; see §2.4), our pipeline is competitive on both quality and efficiency axes.

SEA-RAFT+DA3 — drivetrack/tapvid3d_1022527355599519580_4866_960_2_L58RM2TH_i-3sYbjr6jJQO
solid = predicted, dashed = GT, dot = anchor frame



baseline

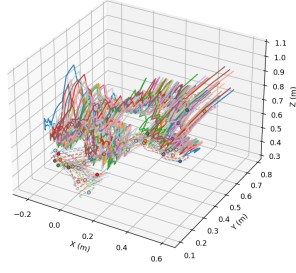
V33 — drivetrack/tapvid3d_1022527355599519580_4866_960_2_L58RM2TH_i-3sYbjr6jJQO
real-metric 3D err: raw=0.73 m, median-scaled=0.74 m (plot scaling=anchor)
solid = predicted, dashed = GT, dot = anchor frame



v33

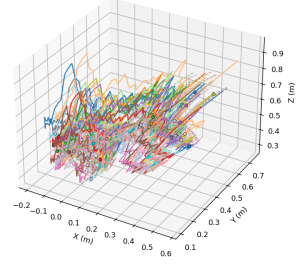
Figure 11: drivetrack (driving, ~ 20 m): the far-field scene where v33's depth correction helps most.

SEA-RAFT+DA3 — adt/Apartment_release_clean_seq140_3
solid = predicted, dashed = GT, dot = anchor frame



baseline

V33 — adt/Apartment_release_clean_seq140_3
real-metric 3D err: raw=0.24 m, median-scaled=0.24 m (plot scaling=anchor)
solid = predicted, dashed = GT, dot = anchor frame



v33

Figure 12: adt (egocentric, ~ 1 m): near-field, where DA3-l is already accurate (tie).

SEA-RAFT+DA3 — drivetrack/tapvid3d_1022527355599519580_4866_960_2_L58RM2TH_i-3sYbjr6jJQO
solid = predicted, dashed = GT, dot = anchor frame

xyt — XY plane vs time

yzt — YZ plane vs time

zxt — Z-X plane vs time

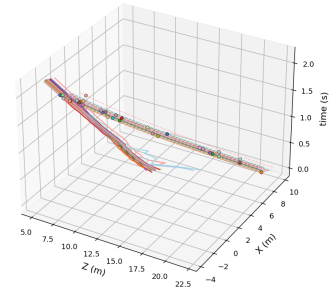
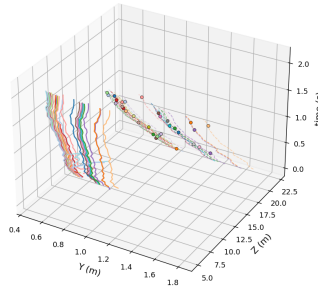
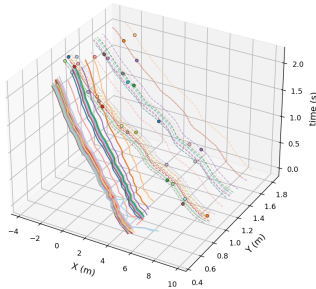


Figure 13: Space-time view (drivetrack baseline): per-axis position over time, $X-t/Y-t/Z-t$, predicted (solid) vs ground truth (dashed).

Table 3: Inference throughput on RTX 4080 (12 GB), measured on TAPVid-3D minival. Methods marked * load depth from a shared DA3-l precompute cache rather than running depth estimation online; SpatialTrackerV2 uses its own internal depth model. Our lightweight SSM refiner adds modest overhead over the baseline while remaining 50–200 $\times$ faster than all competitors.

Method	fps	trainable params
DA3-l depth (shared precompute)*	≈ 10	0
SEA-RAFT + DA3-l (baseline)*	12.4	0
v33 (depth refiner)*	12.5	0.40 M
v35 (DINOv3 + SSM)*, [†]	7.7	0.44 M
v36 (bidir + v35)*	6.2	0.44 M
v37 (WAFT + DA3-l)*	≈ 7.7	0
v39 (WAFT flow + v35)*	≈ 6.3	0.44 M
TAPIP3D*	≈ 3.1	—
SpatialTrackerV2 (s_wind = 60)	≈ 4.0	—
DELTA (DenseTrack3D)*	≈ 7.7	—
TrackCraft3R [§]	≈ 0.06	—

* Loads precomputed DA3-l depth from cache; not counted in fps. The preprocessing runs at ≈ 10 fps on the same GPU and is a one-time offline step shared by all such methods. [†] v35 throughput measured on the drivetrack subset only. [§] TrackCraft3R is a pre-trained 1.3 B-parameter video diffusion model (Wan2.1 DiT + 3 VAEs + LoRA); it is not trained by us, hence “—” for trainable params. Throughput measured via windowed inference (see text).

References

- [1] Yihan Wang, Lahav Lipson, and Jia Deng. SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 36–54, 2024.
- [2] Yihan Wang and Jia Deng. WAF: Warping-Alone Field Transforms for Optical Flow. In *International Conference on Learning Representations (ICLR)*, 2026.
- [3] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth Anything 3: Recovering the Visual Space from Any Views. *arXiv preprint arXiv:2511.10647*, 2025.
- [4] Aakash Sunil Lahoti, Kevin Li, Berlin Chen, Caitlin Wang, Aviv Bick, Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved Sequence Modeling using State Space Principles. In *International Conference on Learning Representations (ICLR)*, 2026.
- [5] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DI-NOv3. *arXiv preprint arXiv:2508.10104*, 2025.
- [6] Skanda Koppula, Ignacio Rocco, Yi Yang, Joe Heyward, Joao Carreira, Andrew Zisserman, Gabriel Brostow, and Carl Doersch. TAPVid-3D: A Benchmark for Tracking Any Point in 3D. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [7] Tuan Duc Ngo, Peiye Zhuang, Chuang Gan, Evangelos Kalogerakis, Sergey Tulyakov, Hsin-Ying Lee, and Chaoyang Wang. DELTA: Dense Efficient Long-range 3D Tracking for Any Video. In *International Conference on Learning Representations (ICLR)*, 2025.
- [8] Bowei Zhang, Lei Ke, Adam W. Harley, and Katerina Fragkiadaki. TAPIP3D: Tracking Any Point in Persistent 3D Geometry. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [9] Jisu Nam, Jahyeok Koo, Soowon Son, Jaewoo Jung, Honggyu An, Junhwa Hur, and Seungryong Kim. TrackCraft3R: Repurposing Video Diffusion Transformers for Dense 3D Tracking. *arXiv preprint arXiv:2605.12587*, 2026.
- [10] Shariq Farooq Bhat, Reiner Birkel, Diana Wofk, Peter Wonka, and Matthias Müller. ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth. *arXiv preprint arXiv:2302.12288*, 2023.
- [11] Yuxi Xiao, Jianyuan Wang, Nan Xue, Nikita Karaev, Yuri Makarov, Bingyi Kang, Xing Zhu, Hujun Bao, Yujun Shen, and Xiaowei Zhou. SpatialTrackerV2: 3D Point Tracking Made Easy. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.